

Rong He

Singapore | hanserong@u.nus.edu | rong.bio

EDUCATION

National University of Singapore

M.Tech. in Software Engineering

Aug 2026 – Oct 2027

Singapore

Korea Advanced Institute of Science and Technology

B.S. in Computer Science — GPA 86/100, top 5% of cohort

Sep 2021 – Aug 2025

Daejeon, Korea

EXPERIENCE

SAIC Motor

AI Agent Developer (Intern)

Feb 2026 – Jul 2026

Shanghai, China

- **Four production agents:** shipped against cost-reduction briefs from internal engineering, product-planning and project-management departments — connector selection, competitive analysis, PRD generation and review, and black-box agent testing — with around 500 internal users across the four.
- **Connector selection:** Flask, LangChain and MongoDB, 8.5k lines of Python. An LLM intent layer decomposes a request into exact match, fuzzy match and range queries, then fans it across five product collections with automatic field-name mapping. Results an engineer confirms are written back to a knowledge base, so equivalent questions reuse a stored template and a human stays in the loop of knowledge capture. Eight releases, including a dataset migration and fallback simplification.
- **Competitive analysis:** an LLM-driven crawler collects public owner reviews of competitor vehicles into MongoDB; six chapter-level agents run as a report pipeline that emits a structured, evidence-backed analysis.
- **AUT:** a black-box agent testing tool, 3.1k lines of Python. Playwright logs in — including a manual handoff for CAPTCHA, SSO and MFA — finds the chat entry, captures the live API call and emits a replayable request template; staged concurrency then runs against automatic stop conditions on success rate, error rate and p95, with probes for prompt injection, cross-session marker leakage and output-format stability. It found the connector agent's degradation point: 100% success and p95 14.2 s at 20 concurrent users, falling to 94.9% at 500, with 51 of 1,000 requests returning empty or malformed output.

Zhengzhou JIEAN Hi-Tech

Algorithm Engineer, Computer Vision (Intern)

Sep 2025 – Jan 2026

Zhengzhou, China

- **Examination system:** built the vision algorithms for a special-operations competency examination run with the municipal emergency-management bureau, where four to five synchronized cameras let models score procedural compliance and completeness in place of manual invigilator grading. It ran live examinations every working day for over a month across three production lines I owned; I also set the annotation specification with the data department so labelling matched the algorithm's assumptions.
- **Graph-based scoring:** replaced "YOLO detection plus hand-written rules" with a graph formulation — YOLOv8 emits each component's centre, size, angle and class, components form a graph under each assembly step's part constraints, and a graph convolutional network predicts installation and step-completion state, recovering the accuracy rules lost by ignoring spatial relations. Models passed internal acceptance at around 90% accuracy, and a fallback of human co-invigilation with tablet scoring meant an algorithm failure never halted an exam in progress.

Shanghai Asia-Pacific MITAC Computer

Software Development Engineer, Java Web (Intern)

Jul 2023 – Aug 2023

Shanghai, China

- **Java Web system:** developed business features and handled day-to-day maintenance inside an existing enterprise system, plus the accompanying front-end pages and a WeChat Mini Program. Tuned the database access layer for peak concurrent traffic — re-sized the connection pool, replaced row-by-row reads and writes with batched operations, simplified queries — after which connection-wait alarms at peak hours stopped firing. The lasting lesson was method rather than numbers: locate where the waiting actually happens before changing any parameter.

PROJECTS

coding-agent-from-scratch — Terminal Coding Agent

Open-source personal project

Sep 2026 – Present

github.com/Darrenus/coding-agent-from-scratch

- **The agent:** a command-line coding assistant written from scratch with no agent framework: a ReAct loop over native tool calling, seven tools, and SEARCH/REPLACE editing behind a four-tier matching chain that absorbs the whitespace and partial-line drift models introduce when quoting code back. Shell runs inside a macOS Seatbelt sandbox whose policy is validated by escape tests rather than by reading it — about 1,300 lines under 52 tests.
- **Repo map:** deciding which code enters context: tree-sitter extracts symbol definitions and references, files are linked into a directed graph, and personalised PageRank ranks them against the task. Constraining edges by actual Python imports raised edge precision from 32% to 100% without losing a real edge; the binding constraint proved to be query understanding rather than ranking, resolved by IDF-weighted seeding. On a 112-file repository the map cut task steps by 21% and input tokens by 33%.
- **Measured, including negative results:** every design decision is settled by a controlled experiment whose harness runs an A/A arm by default, so each effect is reported against its own noise floor — a discipline that retracted one of my own earlier claims. Two textbook agent patterns measured as net losses here: history compression is unnecessary once each tool caps its output, and sub-agent isolation costs 42% more total tokens for 22% less main-loop context with no gain in completion or correctness.

- **Five paths:** a retrieval engine running five paths in parallel — filename and path, section title, tokenized full-text search over SQLite FTS5, dense vectors, and an entity graph — scored under four independent weighting strategies and merged per file, with the per-path score breakdown returned as output rather than hidden behind a debug flag.
- **Weights as a search problem:** every magic number moves into a hot-reloadable configuration; relevance labels come cheaply from which files a model chose to open, recorded with their provenance so weak supervision is never mistaken for human annotation; and NDCG is the objective, chosen because ratio-style metrics are inflated by collapsing all weight onto one path while the ranking gets worse. Grid search maps the space, then a genetic algorithm runs several times independently so the spread across runs separates a finding from noise. Schema, configuration format and ranking metrics are built under 17 passing tests, and the pipeline runs offline with no API key.

Internal Spreadsheet Q&A Platform
Owner — internal SAIC project

Feb 2026 – Apr 2026
Shanghai, China

- **The platform:** turns natural-language questions over uploaded spreadsheets into verified SQL, result tables and charts. A FastAPI and SQLAlchemy backend across 76 modules and 23,331 lines with 56 test files, on MySQL, Redis, MinIO and DashScope (Qwen); a 7,487-line React 19 front end streaming over SSE. Rolled out to 20+ internal pilot users, whose trial feedback estimated roughly 2 hours of manual work removed per user per day.
- **Planner–Executor:** refactored a fixed LLM workflow into a planner that emits only a structured plan — intent, target fields, action sequence, constraints, failure policy — and an executor that runs only whitelisted actions over a read-only SQL layer with table-, field- and permission-level whitelisting, with isolation enforced at the organization and role boundary. Session context, generated SQL and chart configuration persist so follow-up questions resolve against prior turns.
- **Patent and rewrite:** co-authored the invention disclosure behind the design (filed Jun 2026, under examination), defining the pre-execution gating metric $S = 0.35F + 0.25A + 0.20C + 0.10P + 0.10R$ where access compliance A is a hard gate. Later re-architected the core as a native tool inside an open-source agent runtime, where the model sees only a controlled `clarify / result_table / result_chart` protocol.

RESEARCH

Preprints

Rong He. **Decomposition Buys Integrity, Not Yield.** [arXiv:2609.17464](https://arxiv.org/abs/2609.17464) [cs.MA], Sep 2026.

Splitting a task across a tree of sub-agents buys integrity — each agent sees a smaller context — but costs yield, the share of what the leaves discover that survives the trip back to the root. Models retention as $r(b) = 1/b$, validates it over 20,000 irregular trees, and recovers an exponent of $\delta = 0.34$ on 600 production deep-research traces. Across 743,819 production tool calls only 0.7–11.3% of sessions justify delegation against the 7.8% that actually occurs: flat is optimal for yield, and decomposition earns its keep on root-context exposure and cost instead.

Rong He. **Prefix Sharing Is a Sorting Problem.** [arXiv:2609.13692](https://arxiv.org/abs/2609.13692) [cs.DS], Sep 2026, 15 pp.

LLM serving reuses KV cache by exact prefix match, so the order in which reusable pieces — retrieved passages, tool definitions, few-shot exemplars — are assembled into a prompt decides how much computation can be shared. Every deployed system fixes that order by one global convention, which this proves optimal only for two-piece requests. Choosing chunk orders is choosing a binary hierarchy over requests, making the leave-one-out optimum the merge-sort recursion, so a global order pays $\Theta(n^2)$ against a true cost of $\Theta(n \log n)$; on BM25 traces over three BEIR corpora the layout cuts prefill by 17–36% against production RAG ordering.

Rong He. **Sharp Limits on Time-Reversal Detection in Binary Hidden Markov Models.** Zenodo, 2026. DOI: [10.5281/zenodo.22684760](https://doi.org/10.5281/zenodo.22684760)

Constructs positive stationary binary HMMs whose observations hide their arrow of time at any $2n - 4$ instants, and proves the matching detection scale $\Theta(|\theta|^{-(4n-6)})$ sharp.

Patent

An auditable planning-and-execution method and system for natural-language querying of spreadsheets. Filed Jun 2026, under examination.

SKILLS

Programming	Python, TypeScript, Go, Java, C++, SQL
Agents & LLM	Planner–executor runtimes, tool use, memory and context, multi-agent, MCP, LangGraph
RAG	Hybrid retrieval over full text, dense vectors and graphs; context assembly, KV-cache reuse
Evaluation	Evaluation-set design, ranking metrics, ablation and parameter search, adversarial testing
Distributed	Service backends, async task pipelines and queues, SSE streaming, role-based isolation, CI
Data	MySQL, MongoDB, Redis, SQLite, MinIO, Chroma; schema profiling and ingestion pipelines
Vision	Object detection and pose estimation, graph neural networks, GPU inference deployment
Languages	Mandarin (native), English (TOEFL 90), Korean (TOPIK Level 2)

HONOURS

Top 10, Nowcoder AI Coding Challenge Cup (2026) | Merit Award, “Guoji Cup” Industrial Intelligent Agent Competition (2026) | Honorable Mention, COMAP MCM (2024)